

Question 1 of 21

A loading indicator is useful but disappears after response. What kind of state is it?

- ☐ Temporary frontend state
- ☐ Persistent frontend preference
- ☐ Temporary backend processing state
- ☐ Durable database state

Question 2 of 21

User is authenticated but lacks permission to delete another user's reminder. Which status commonly fits?

☒ 201 Created

☐ 403 Forbidden

☐ 404 Created

☐ 503 Service Unavailable

Question 3 of 21

API is temporarily unavailable, but frontend still collects a booking request. What should frontend show?



Guaranteed success



Retryable or unavailable outcome



Random driver assignment



Database administration screen

Question 4 of 21

A response includes `Content-Type: text/html`. What does this header describe?

☒ Format of response content

☐ Method used by client

☐ Permission of current actor

☐ Status family

Question 5 of 21

Server returns accepted order ID and confirmed total. Why are these response payload fields?

☐ Client proposes them before validation

☐ Provider returns them after processing

☐ They are HTTP methods

☐ They are query parameters

Question 6 of 21

Which question best diagnoses a failed state change?



Did the frontend display the intended value at any point?



Did the database contain a value before the request began?



Which component changed, which did not, and what was confirmed?



Should the frontend immediately retry the same operation?

Question 7 of 21

Which information should a ride-booking API normally expose to frontend?

- ☐ Database password
- ☐ Every internal server variable
- ☐ Ride confirmation or explained failure
- ☐ Complete backend source code

Question 8 of 21

Why must frontend, backend, and database communicate?

☐ Every component must duplicate all data

☐ Every component must use same programming language

☐ Separate responsibilities need information from one another

☐ Frontend must control every decision

Question 9 of 21

Which statement correctly distinguishes missing key and null?

- ☐ They always mean same thing
- ☐ Missing key is boolean false
- ☐ null explicitly states no value; missing omits field
- ☐ null is invalid JSON

[< Previous](#)



Search



Question 10 of 21

Weather response contains `"alerts": ["rain", "wind"]`. What is `alerts` value?

☐ Array

☐ Object

☐ String

☐ null

Question 11 of 21

Which chain preserves correct component responsibilities?



Database draws screen, frontend validates permission, backend stores permanently



Frontend persists order, database authenticates actor, API draws button



Backend draws screen, frontend exposes tables, database sends user input



Frontend collects, backend validates, database persists

Question 12 of 21

Browser cannot establish network connection and receives no response. Which statement is correct?

☐ No HTTP status code arrived

☐ Server definitely returned 500

☐ Client definitely received 404

☐ Request definitely succeeded

Question 13 of 21

Profile has name, city, and alerts. User changes only alerts. Which method best expresses partial modification?

☐ GET

☐ PATCH

☐ DELETE

☐ POST

Question 14 of 21

The user taps Add to Cart, and the badge changes from 0 to 1. Which statement is most accurate?

☐ The tap is permanent database state

☐ The new badge value is an event rather than state

☐ The database definitely changed before the screen

☐ The tap is an event that triggered a frontend state transition

Question 15 of 21

Which complete interpretation is correct?

☐ GET means create; 201 means client error

☐ PATCH means partial modification; 2xx means success

☐ DELETE means retrieve; 404 means success

☐ PUT means response format; 5xx means redirection

Question 16 of 21

A weather frontend asks server for Pune forecast without requesting a state change. Which method fits?

☐ GET

☐ POST

☐ PATCH

☐ DELETE

Which statement correctly relates API and HTTP?

☐ HTTP can carry web API requests and responses

☐ API and HTTP are identical terms

☐ Every API is database table

☐ HTTP is user interface

Question 18 of 21

Which method expresses replacing complete target representation?

☐ POST

☐ PATCH

☐ PUT

☐ GET

Question 19 of 21

What is primary responsibility of response in a request-response exchange?

☒ Start request from client

☐ Store all application data

☐ Replace backend validation

☐ Report outcome to caller

Which sequence best describes a confirmed saved-cart update?

☐ Frontend requests → database stores directly
→ backend is notified

☐ Frontend requests → backend confirms →
database attempts the write

☐ Backend validates → frontend confirms →
database stores later

☐ Frontend requests → backend validates →
database stores → response confirms

Question 21 of 21

ParkEasy is an app for finding and reserving parking. A signed-in driver opens a parking lot's page, picks an open spot, and taps **Reserve**. For the reservation to be useful later, the spot must be checked to confirm it is still free and then stored safely, and the driver must see a clear confirmation on screen.

Three parts of the system cooperate to make this happen:

- the **App** the driver is using (shows the spot and collects the tap),
- a backend **Reservation Service** that owns the reservation rules (checks the spot is still free and coordinates saving), and
- the **Database** where the confirmed reservation is stored durably.

The driver tapping **Reserve** is the **start**. A confirmation that the spot is reserved is the **visible end**.

Task

Draw the correct **UML sequence diagram** of this Reserve a Spot interaction. You are **not** given the messages — work them out from the scenario. As you build it, reason through four questions:

1. **Who takes part?** Show the driver as an external **actor**, and the App, the Reservation Service, and the Database as **participants**, each with its own dashed **lifeline**. Decide what each part is responsible for. (*Hint: a message arrow already shows communication, so you do not need a separate "Network" lifeline, and the Database is a participant, not an actor.*)
2. **What is the right order?** Each request should reach the part that can actually perform that step. The driver should **never** message the Database directly.
3. **How does the driver find out it worked?** Make sure the result travels back through the same parts until the driver sees a **reserved spot**.
4. **When is each part busy, and what kind of arrow is this?** Use **activation bars** to show real work, and use **different line styles** for requests and returned results.

Use standard, simple notation. **Time must run from the top of the diagram downward**. Do **not** add code, class diagrams, database schemas, screen mock-ups, or advanced UML fragments.

Plan Before Drawing

Write the interaction as a short list of ordered lines first, then draw from it:

Sender → Receiver: what is being asked or returned
--

Check your plan before you draw:

- Does each request point to the part that performs the work?
- Does the result travel back through the Reservation Service and App to the driver?
- Does the Database return to the **Reservation Service**, not straight to the driver?
- Does the last line match the visible result: the driver sees the reserved spot?

Create It in draw.io

1. Open draw.io and create a blank diagram.
2. Choose **More Shapes** (bottom of the left panel) and enable **UML** or **UML 2.5**.
3. Place the actor and the three participants across the top, then add their dashed lifelines.
4. Use **solid** connectors for requests and **dashed** connectors for returned results.
5. Add **narrow activation bars** only where a part is actually working.
6. Keep every label readable, then export the finished diagram.

Submission Requirements

- Submit **exactly one PNG image** of the finished sequence diagram (exported from draw.io: **File** → **Export as** → **PNG**).
- Make sure the image is **readable at normal zoom** — every participant name and message label must be legible.
- Use the same part names throughout (equivalents are fine: App/Frontend, Reservation Service/Backend, DB).
- **PNG only.** Do not submit a .drawio source file, a PDF, a link, or a photo of a screen.

Submission Requirements

- Submit **exactly one PNG image** of the finished sequence diagram (exported from draw.io: **File** → **Export as** → **PNG**).
 - Make sure the image is **readable at normal zoom** — every participant name and message label must be legible.
 - Use the same part names throughout (equivalents are fine: App/Frontend, Reservation Service/Backend, DB).
 - **PNG only.** Do not submit a `.drawio` source file, a PDF, a link, or a photo of a screen.
-